

$O(n \log n)$, N-Log-N Time

Sorting a set of books into alphabetical order is an $n \log n$ time operation. This order is the runtime of $O(n)$ and $O(\log n)$ multiplied together. You can think of a $O(n \log n)$ task as a $O(\log n)$ task that must be performed n times. Here's a casual description of why.

Start with a stack of books to alphabetize and an empty bookshelf. Follow the steps for a binary search algorithm, as detailed in “ $O(\log n)$, Logarithmic” on page 232, to find where a single book belongs on the shelf. This is an $O(\log n)$ operation. With n books to alphabetize, and each book taking $\log n$ steps to alphabetize, it takes $n \times \log n$, or $n \log n$, steps to alphabetize the entire set of books. Given twice as many books, it takes a bit more than twice as long to alphabetize all of them, so $n \log n$ algorithms scale fairly well.

In fact, all of the efficient general sorting algorithms are $O(n \log n)$: merge sort, quicksort, heapsort, and Timsort. (Timsort, invented by Tim Peters, is the algorithm that Python's `sort()` method uses.)

$O(n^2)$, Polynomial Time

Checking for duplicate books on an unsorted bookshelf is a polynomial time operation. If there are 100 books, you could start with the first book and compare it with the 99 other books to see whether they're the same. Then you take the second book and check whether it's the same as any of the 99 other books. Checking for a duplicate of a single book is 99 steps (we'll round this up to 100, which is our n in this example). We have to do this 100 times, once for each book. So the number of steps to check for any duplicate books on the bookshelf is roughly $n \times n$, or n^2 . (This approximation to n^2 still holds even if we were clever enough not to repeat comparisons.)

The runtime increases by the increase in books squared. Checking 100 books for duplicates is 100×100 , or 10,000 steps. But checking twice that amount, 200 books, is 200×200 , or 40,000 steps: four times as much work.

In my experience writing code in the real world, I've found the most common use of big O analysis is to avoid accidentally writing an $O(n^2)$ algorithm when an $O(n \log n)$ or $O(n)$ algorithm exists. The $O(n^2)$ order is